UNIVERSIDADE FEDERAL DE ALFENAS — UNIFAL-MG
Bacharelado em Ciência da Computação

Carlos Eduardo Pereira dos Santos
Matrícula: 2024.2.08.001

ALOCAÇÃO MULTIOBJETIVO DE MICROSSERVIÇOS: IMPLEMENTAÇÃO E AVALIAÇÃO DO MOMA

Alfenas — MG
2026

Carlos Eduardo Pereira dos Santos
Matrícula: 2024.2.08.001

ALOCAÇÃO MULTIOBJETIVO DE MICROSSERVIÇOS: IMPLEMENTAÇÃO E AVALIAÇÃO DO MOMA

Relatório da segunda etapa do trabalho final apresentado à Universidade Federal de Alfenas — UNIFAL-MG, no curso de Bacharelado em Ciência da Computação, como parte da avaliação da disciplina DCE770 — Heurísticas e metaheurísticas.

Professor: Iago Augusto de Carvalho.

Alfenas — MG
2026

RESUMO

Este trabalho apresenta a implementação e a avaliação de uma adaptação do algoritmo Multi-Objective Microservice Allocation (MOMA), proposto por Chen e Wen para alocação de microsserviços em nuvens heterogêneas. A abordagem utiliza o NSGA-II para minimizar a carga ponderada dos nós e o custo de comunicação, respeitando limites de CPU, memória e réplicas. A implementação em Python é acompanhada de hipóteses explícitas para aspectos não completamente especificados no artigo. A validação utiliza uma instância pequena com fronteira de Pareto exata, recuperada em vinte execuções. A avaliação principal emprega seis instâncias híbridas das aplicações EdgeX Foundry, Sock Shop e Kubeflow, com demandas publicadas de CPU e memória e infraestrutura e rede sintéticas. Foram realizadas 240 execuções de quatro métodos, com dez sementes e 1.640 avaliações por execução. O MOMA apresentou hipervolume médio superior ao da busca aleatória nos seis cenários, mas desempenho misto frente a dois gulosos locais. Os resultados evidenciam limitações da adaptação e não estabelecem equivalência com os ganhos percentuais reportados no ambiente original do artigo.

Palavras-chave: alocação de microsserviços; otimização multiobjetivo; algoritmo genético; NSGA-II; nuvem heterogênea.

ABSTRACT

This work presents the implementation and evaluation of an adaptation of the Multi-Objective Microservice Allocation (MOMA) algorithm proposed by Chen and Wen for microservice placement in heterogeneous clouds. The approach uses NSGA-II to minimize weighted node load and communication cost while respecting CPU, memory, and replica constraints. The Python implementation explicitly documents assumptions for aspects not fully specified in the paper. Validation uses a small instance with an exact Pareto front, recovered in twenty runs. The main evaluation uses six hybrid instances based on EdgeX Foundry, Sock Shop, and Kubeflow, combining published CPU and memory demands with synthetic infrastructure and networks. A total of 240 runs of four methods were performed, using ten seeds and 1,640 evaluations per run. MOMA achieved higher average hypervolume than random search in all six scenarios, but mixed performance against two local greedy methods. The results highlight limitations of the adaptation and do not establish equivalence with the percentage improvements reported in the original paper's experimental environment.

Keywords: microservice allocation; multiobjective optimization; genetic algorithm; NSGA-II; heterogeneous cloud.

SUMÁRIO

1 INTRODUÇÃO 5

2 RESUMO DO ARTIGO ORIGINAL 6

3 IMPLEMENTAÇÃO 8

4 VALIDAÇÃO E PROTOCOLO EXPERIMENTAL 9

5 RESULTADOS LOCAIS 10

6 COMPARAÇÃO COM OS RESULTADOS DO ARTIGO 12

7 CONCLUSÕES 14

REFERÊNCIAS 15

1 INTRODUÇÃO

A alocação de microsserviços em infraestrutura heterogênea envolve compromissos entre o uso de recursos e a comunicação entre componentes. Chen e Wen (2024) propõem o algoritmo Multi-Objective Microservice Allocation (MOMA), fundamentado no NSGA-II, para tratar esses objetivos em ambientes de nuvem baseados em contêineres.

Este trabalho apresenta uma implementação acadêmica adaptada do MOMA e avalia seu comportamento em instâncias locais. O objetivo é implementar o núcleo da meta-heurística, verificar a consistência dos cálculos e contrastar os resultados obtidos com as conclusões do artigo. As hipóteses necessárias para os dados não publicados são identificadas explicitamente, delimitando o alcance da comparação.

O relatório retoma o resumo da primeira etapa, descreve a implementação e a validação, apresenta o protocolo e os resultados experimentais e discute as limitações de reprodução dos ganhos reportados pelos autores.

2 RESUMO DO ARTIGO ORIGINAL

A crescente utilização de arquiteturas de microsserviços e tecnologias de conteinerização tem ampliado a necessidade de mecanismos eficientes para o gerenciamento de recursos computacionais em ambientes de nuvem. Nesse contexto, a distribuição de microsserviços entre diferentes nós e clusters representa um problema complexo, especialmente quando a infraestrutura apresenta características heterogêneas. O artigo *Optimal Resource Allocation Using Genetic Algorithm in Container-Based Heterogeneous Cloud*, de Chen e Wen (2024), investiga esse problema e propõe o algoritmo Multi-Objective Microservice Allocation (MOMA), baseado em uma versão aprimorada do algoritmo genético NSGA-II. O estudo busca otimizar a alocação de microsserviços considerando simultaneamente a utilização de recursos, a sobrecarga de comunicação de rede e aspectos relacionados à confiabilidade da infraestrutura.

A motivação da pesquisa está relacionada às limitações dos mecanismos tradicionais de escalonamento. Segundo os autores, o algoritmo padrão do Kubernetes concentra-se na utilização dos recursos físicos, mas não considera adequadamente os custos de transmissão de dados e a confiabilidade dos nós. Além disso, muitos trabalhos anteriores concentram-se em ambientes homogêneos ou não contemplam conjuntamente as características de múltiplos clusters heterogêneos. Essas limitações dificultam a distribuição eficiente das aplicações, podendo provocar desequilíbrios de carga, aumento da comunicação entre componentes e maior exposição a falhas. Como a alocação de recursos em contêineres é tratada como um problema NP-difícil, os autores recorrem a técnicas de otimização meta-heurística para buscar soluções aproximadas eficientes.

A principal contribuição do trabalho consiste no desenvolvimento do MOMA, um algoritmo genético multiobjetivo adaptado ao posicionamento de microsserviços em ambientes heterogêneos. O modelo de otimização considera dois objetivos principais. O primeiro procura minimizar a métrica de balanceamento de recursos, levando em conta a utilização de CPU e memória nos nós dos clusters. O segundo busca reduzir a sobrecarga de comunicação de rede, considerando as interações entre microsserviços, as distâncias entre os nós e as taxas de falha. O modelo também estabelece restrições relacionadas à capacidade computacional disponível, de modo que os requisitos de CPU e memória dos microsserviços sejam atendidos durante a alocação.

O algoritmo MOMA é desenvolvido com base no NSGA-II e utiliza mecanismos evolutivos para explorar diferentes configurações de posicionamento. A representação dos indivíduos é realizada por meio de cromossomos que descrevem a distribuição dos microsserviços entre os nós. O processo evolutivo inclui seleção por torneio binário, cruzamento *Simulated Binary Crossover* (SBX) e operadores de mutação por inserção e remoção. A seleção da próxima geração considera a ordenação não dominada e a distância de

aglomeração (*crowding distance*), mecanismos utilizados pelo NSGA-II para preservar soluções não dominadas e promover diversidade na população. Como o problema possui dois objetivos, o resultado é representado por uma fronteira de Pareto, que reúne soluções com diferentes compromissos entre balanceamento de recursos e custos de comunicação. A qualidade das soluções é avaliada por meio da métrica hipervolume.

Além do algoritmo, os autores apresentam uma estrutura composta por quatro unidades: monitoramento, análise de dados, otimização de posicionamento e gerenciamento Kubernetes. O monitoramento coleta informações sobre consumo de CPU e memória, latência e vazão dos microsserviços. Os dados são analisados e armazenados para orientar a etapa de otimização, enquanto a unidade de gerenciamento utiliza a API do Kubernetes para implantar os serviços nos nós selecionados. A avaliação experimental utiliza três aplicações: Kubeflow, Sock Shop e EdgeX Foundry, executadas em uma infraestrutura formada por clusters com diferentes configurações de hardware. Foram utilizados Prometheus e Grafana para monitoramento, Locust para geração de carga e ferramentas de otimização multiobjetivo para implementação e avaliação do algoritmo. Os autores também comparam o MOMA com o algoritmo padrão do Kubernetes, o Multiopt e uma heurística Greedy.

Os resultados experimentais indicam que o MOMA apresentou melhorias nos três aspectos analisados: utilização de recursos, sobrecarga de comunicação e confiabilidade. Em comparação com o escalonador padrão do Kubernetes, foram observadas melhorias na utilização dos recursos nas três aplicações. Para dois usuários, por exemplo, os ganhos reportados na utilização dos recursos do cluster foram de 11,1% no EdgeX Foundry, 11,3% no Sock Shop e 7,0% no Kubeflow. Com oito usuários, os ganhos correspondentes foram de 7,0%, 10,0% e 5,3%. Em relação à comunicação, o MOMA apresentou resultados favoráveis, especialmente em cenários com maior quantidade de solicitações. A avaliação de confiabilidade também mostrou vantagens em relação aos métodos comparados, que não consideravam as taxas de falha dos nós da mesma maneira.

Conclui-se que o MOMA constitui uma abordagem para a alocação multiobjetivo de microsserviços em clusters Kubernetes heterogêneos, integrando informações sobre recursos, comunicação e falhas. Os experimentos apresentados pelos autores demonstram melhorias nos indicadores avaliados para as três aplicações estudadas. Entretanto, o trabalho também aponta possibilidades de investigação futura, incluindo o gerenciamento explícito de GPUs, a ampliação do número de clusters e microsserviços avaliados, a inclusão de métricas adicionais, como consumo energético e saúde dos serviços, e a comparação com outras meta-heurísticas. Essas possibilidades indicam caminhos para ampliar a avaliação e a aplicabilidade da abordagem em infraestruturas de nuvem e computação de borda.

3 IMPLEMENTAÇÃO

A implementação foi desenvolvida em Python, utilizando somente a biblioteca padrão. O núcleo evolutivo usa o NSGA-II: seleção por torneio binário, cruzamento SBX, mutação por inserção e remoção, ordenação não dominada, crowding distance e seleção elitista da união de pais e descendentes. O resultado é o conjunto de alocações viáveis não dominadas da população final.

Cada solução é uma matriz inteira: a posição X[i][h] informa quantas réplicas do serviço i estão no nó h. A modelagem permite várias réplicas no mesmo nó, conforme os exemplos das Figuras 4 e 6 do artigo. Cada réplica tem consumo fixo de CPU e memória; todas devem respeitar os limites de contagem e a capacidade de cada nó. Soluções viáveis têm preferência sobre inviáveis. Entre inviáveis, a seleção considera o menor excesso normalizado das restrições.

Os dois objetivos são minimizados. O primeiro é a maior carga ponderada de CPU e memória entre os nós, com pesos calculados pelos desvios padrão populacionais das razões de uso em cada cluster. Na ausência de variação, são usados pesos iguais. O segundo soma volume de comunicação, distância e fator de falha do nó emissor, com o fator (1 + falha) da formulação original. O tráfego é repartido uniformemente entre as réplicas e o custo dentro do mesmo nó é zero.

Essas definições resolvem lacunas do artigo e constituem adaptações explícitas. Também foram definidos arredondamento do SBX para inteiros, reparo dos limites de réplicas e inclusão da comunicação entre clusters. As ambiguidades e equações operacionais estão detalhadas em modelo_operacional.md (arquivo ../docs/modelo_operacional.md). A implementação não executa a infraestrutura de monitoramento e implantação Kubernetes.

4 VALIDAÇÃO E PROTOCOLO EXPERIMENTAL

Antes dos experimentos maiores, foi criada uma instância sintética de dois nós e dois serviços, com contas manuais. A enumeração identificou 25 alocações e 11 soluções viáveis. Sua fronteira exata contém os pontos (0,50; 30,90) e (0,75; 0), que expressam o compromisso entre carga e comunicação. O MOMA recuperou ambos os pontos em 20 execuções com sementes de 0 a 19, população e descendentes 20 e 20 gerações. Foram aprovados 39 testes automatizados do modelo, dos operadores, das métricas e do procedimento experimental. A validação didática verifica o funcionamento; não estabelece qualidade ótima em instâncias maiores.

Na bateria principal, foram usadas seis instâncias híbridas: EdgeX Foundry, Sock Shop e Kubeflow, com demandas de um e dez usuários. CPU e memória foram transcritas das Tabelas 4 e 5; a rede, a infraestrutura e os limites de réplicas são sintéticos. Há seis nós em três clusters, com capacidades heterogêneas e uma a duas réplicas por serviço. As demandas correspondem a nove serviços no EdgeX e 14 em cada uma das outras aplicações. O tráfego sintético cresce linearmente com a carga e não utiliza os valores agregados de Interaction do artigo.

Foram comparados quatro métodos: MOMA adaptado, busca aleatória, guloso de recursos e guloso de comunicação. Os gulosos inserem as réplicas mínimas, selecionando o nó por prioridades lexicográficas e variando a ordem dos serviços nos reinícios. São comparadores locais; não são implementações dos comparadores Kubernetes, Multiopt e Greedy utilizados pelos autores.

A configuração do MOMA foi população 40, descendentes 40, 40 gerações, cruzamento 0,5, mutação do par 0,3 e índice SBX 10. Cada método recebeu 1.640 avaliações por execução, incluindo candidatos repetidos, inviáveis e parciais. As sementes foram de 0 a 9 em cada instância, totalizando 240 execuções e 393.600 avaliações de busca. O ambiente utilizou Python 3.14.8. Esses tamanhos e orçamento são menores que os parâmetros publicados no artigo. As decisões foram documentadas antes da conclusão da bateria, sem ajuste para favorecer os resultados observados. O protocolo experimental (arquivo ../docs/ protocolo_experimental.md) registra as regras de geração, o orçamento e as diferenças entre os métodos.

5 RESULTADOS LOCAIS

A qualidade foi avaliada por hipervolume (HV), com referência normalizada (1.1, 1.1). O objetivo de recursos está entre zero e um nas soluções viáveis; a comunicação é dividida por um limite conservador calculado a partir dos volumes, distâncias e falhas da instância. Todos os métodos e sementes de uma instância usam a mesma escala. HV maior indica melhor cobertura da região de interesse. O HV não é percentual de utilização nem prova de otimalidade.

A Tabela 1 apresenta média e desvio padrão amostral das dez execuções. As comparações devem ser realizadas dentro de cada linha, pois as instâncias têm dados e escalas diferentes.

Tabela 1 – Hipervolume médio e desvio padrão amostral em dez execuções

Aplicação / usuários	MOMA	Aleatório	Guloso de recursos	Guloso de comunicação
EdgeX Foundry / 1	0,903656 ± 0,038625	0,778961 ± 0,047516	0,795024 ± 0,064681	1,042075 ± 0,009795
EdgeX Foundry / 10	0,480974 ± 0,053074	0,385372 ± 0,033309	0,552266 ± 0,032915	0,583480 ± 0,028650
Sock Shop / 1	0,683824 ± 0,050744	0,552319 ± 0,021313	0,629872 ± 0,026168	0,918673 ± 0,010426
Sock Shop / 10	0,431243 ± 0,057186	0,288766 ± 0,025509	0,476678 ± 0,018569	0,506685 ± 0,046551
Kubeflow / 1	0,529781 ± 0,037570	0,462769 ± 0,022527	0,614071 ± 0,029378	0,699038 ± 0,014647
Kubeflow / 10	0,343800 ± 0,059698	0,280649 ± 0,021420	0,368448 ± 0,024277	0,342875 ± 0,015700

Fonte: elaboração própria (2026).

Todas as 240 execuções encontraram ao menos uma solução viável. Em média, o MOMA superou a busca aleatória nos seis cenários. Nas comparações por semente, obteve HV maior em 56 de 60 pares. Frente ao guloso de recursos, apresentou média superior em dois cenários, EdgeX e Sock Shop com um usuário, e inferior nos outros quatro. Foram 24 vitórias e 36 derrotas nas comparações por semente.

O guloso de comunicação apresentou média superior ao MOMA em cinco cenários. No Kubeflow com dez usuários, as médias foram próximas: 0,343800 para o MOMA e 0,342875 para o guloso, com cinco vitórias para cada método nas comparações por semente.

Considerando os seis cenários, o MOMA venceu o guloso de comunicação em seis pares e perdeu em 54. Não foram realizados testes de significância; portanto, essas contagens e médias têm caráter descritivo.

Os resultados indicam vantagem da evolução sobre a amostragem aleatória nesse conjunto de instâncias, mas não uma vantagem geral frente aos gulosos locais. O custo fixo por réplica, a presença de comunicação local de custo zero, a exploração adicional de réplicas pelo MOMA e a discretização do SBX são hipóteses plausíveis para diferenças observadas. Não foram executadas ablações para isolar a contribuição de cada fator. A hipótese de tráfego uniforme e a escolha de um único grafo por aplicação também limitam a generalização.

As alocações, os objetivos, os tempos e as métricas por execução estão em resultados/experimentos (arquivo ../resultados/experimentos). O arquivo analise.md (arquivo ../resultados/experimentos/analise.md) contém as tabelas completas, incluindo os tempos e as comparações por semente. A busca exata não foi executada nessas seis instâncias maiores; sua fronteira ótima permanece desconhecida.

6 COMPARAÇÃO COM OS RESULTADOS DO ARTIGO

Chen e Wen (2024) reportam melhorias de utilização, comunicação e confiabilidade frente aos comparadores do artigo. Para dois usuários, os ganhos de utilização reportados frente ao Kubernetes foram de 11,1% no EdgeX, 11,3% no Sock Shop e 7,0% no Kubeflow; com oito usuários, foram de 7,0%, 10,0% e 5,3%, respectivamente. Esses percentuais se referem à avaliação dos autores e não foram calculados pela implementação deste trabalho.

O Quadro 1 sintetiza as diferenças entre os dois estudos.

Quadro 1 – Condições de avaliação no artigo e neste trabalho

Aspecto	**Artigo original**	**Experimentos deste trabalho**
Demandas	Medições das aplicações e cargas de um a dez usuários	CPU e memória publicadas para um e dez usuários
Infraestrutura e rede	Ambiente Kubernetes heterogêneo e parâmetros dos autores	Infraestrutura e grafo sintéticos, com hipóteses documentadas
Comparadores	Kubernetes, Multiopt e Greedy	Aleatório, guloso de recursos e guloso de comunicação locais
Configuração	População e descendentes 200 na Tabela 3; critério 25.000	População e descendentes 40; 40 gerações
Resultados avaliados	Utilização, sobrecarga e requisições falhas; uso de HV na avaliação de soluções	Objetivos simulados, HV normalizado, viabilidade e tempo
Conclusão de desempenho	Vantagens reportadas frente aos três comparadores	Superior ao aleatório em média; desempenho misto frente aos gulosos

Fonte: elaboração própria (2026), com base em Chen e Wen (2024).

A vantagem ampla reportada pelos autores não foi demonstrada nesta adaptação. O resultado local é melhor que o aleatório nos seis cenários em termos de HV médio, pior que os gulosos em vários cenários e numericamente próximo ao guloso de comunicação no Kubeflow com dez usuários. Essa classificação vale apenas para os comparadores e as métricas locais. Os dados disponíveis e as diferenças de configuração não permitem determinar se os resultados da implementação são melhores, piores ou equivalentes aos percentuais do artigo em condições idênticas. Essa ausência de equivalência é uma limitação da comparação exigida nesta etapa.

Não foram medidas latência, vazão ou requisições falhas. O uso de taxas de falha na função de comunicação não equivale a uma avaliação experimental de confiabilidade. Para uma comparação numérica direta, seriam necessários dados compatíveis de comunicação, regras de réplicas e carga, infraestrutura, comparadores e orçamento, além de resolver as inconsistências entre texto, tabela e pseudocódigo do artigo.

7 CONCLUSÕES

Foi implementado um MOMA adaptado, acompanhado de instâncias, instruções de execução, testes e resultados reproduzíveis. A validação exata da instância pequena e a auditoria das soluções conferem consistência à implementação do modelo operacional adotado. Os experimentos maiores mostram melhoria média frente à busca aleatória e limitações frente a métodos construtivos simples.

O trabalho evidencia a importância de explicitar hipóteses de modelagem e comparar métodos sob um orçamento registrado. Os próximos estudos podem avaliar outros grafos e tamanhos de população, executar ablações dos operadores e obter medições de tráfego e atendimento por réplica. A reprodução dos ganhos originais permanece condicionada à disponibilidade de dados e configurações equivalentes.

REFERÊNCIAS

CHEN, Qi-Hong; WEN, Chih-Yu. Optimal resource allocation using genetic algorithm in container-based heterogeneous cloud. *IEEE Access*, v. 12, p. 7413–7429, 2024. DOI: https://doi.org/10.1109/ACCESS.2024.3351944. Disponível em: https://doi.org/10.1109/ACCESS.2024.3351944. Acesso em: 4 nov. 2026.